

System Architecture for OpenCodeReasoning Agent Diagnostic Challenge

An LLM-Finetuning challenge using
Qwen-coder-1.5B

Santhosh Muthukrishnan



Introduction

- OpenCodeReasoning (OCR) is a popular challenge that attempts to address the limitation of LLMs to generate logically correct code.
- The challenge has been explored in several aspects of code correctness - logical explanation, debugging, etc.
- This variant of the challenge focuses on **debugging** buggy code

Dataset

- ⬡ The data for this challenge comes from the OpenCodeReasoning2 dataset hosted by Nvidia in HuggingFace Hub at <https://huggingface.co/datasets/nvidia/OpenCodeReasoning-2>
- ⬡ The data is built by leveraging two independent LLMs in a generator-critic setting
 - **Deepseek's R1** acts as the generator. It provides a CoT and code to solve the question
 - **Alibaba's QwQ** provides critique for the code that R1 had generated
- ⬡ Includes problem statements from 4 external datasets.

Dataset - OpenCodeReasoning2

- ⬡ The OpenCodeReasoning2 data consists of:
 - A question (initially empty, needs to be manually filled)
 - CoT generated by R1 to attempt solving the problem
 - Code generated by R1 in the above attempt
 - Critique by QwQ on R1's code ← (Our key column)
 - Dataset name (The external dataset from which the problem was taken)
 - Difficulty
 - Judgement (Is the code correct according to QwQ?)
- ⬡ Only code with Judgement='wrong' are considered (buggy code)

Dataset - Taco

- ⬡ This challenge focuses on Python questions marked 'Easy' for difficulty. Such entries in the OpenCodeReasoning2 database are only associated with the 'Taco' external database.
- ⬡ The Taco database consists of:
 - Question statement
 - Objectively correct solutions list
 - Sample inputs and outputs

Concerns to keep in mind

- ⬡ The dataset itself is created by LLM-generated content, which has a chance to be wrong or deviated from the truth. This introduces a **bias** element when used to train other LLMs.
 - This especially affects reliability as **Judgement** is used to filter
- ⬡ Qwen-**1.4B** is too small for such a complex task involving deep logical reasoning
- ⬡ Due to time and usage quota constraints, **only a certain number of steps** of each fine-tuning operation is done.
- ⬡ **Since the dataset has no “starter code - corrected code” pairs, the challenge takes a different approach of using one buggy code and a random correct code.**

Two Aspects of OCR Challenge

Provide Analysis

This aspect is tackled through Supervised Fine-tuning (SFT)

- ⬡ SFT establishes **baseline behavior**. GRPO refines it using **reward-driven optimization**

Provide correct working code

This aspect is tackled through Reinforcement Learning (RL) with Group-Relative Policy Optimisation (GRPO)

Execution Plan

⬡ Data Preprocessing:

- Filter data rows to match challenge constraints (easy, python)
- Combine OCR2 and Taco datasets
- Now the questions column is enumerated

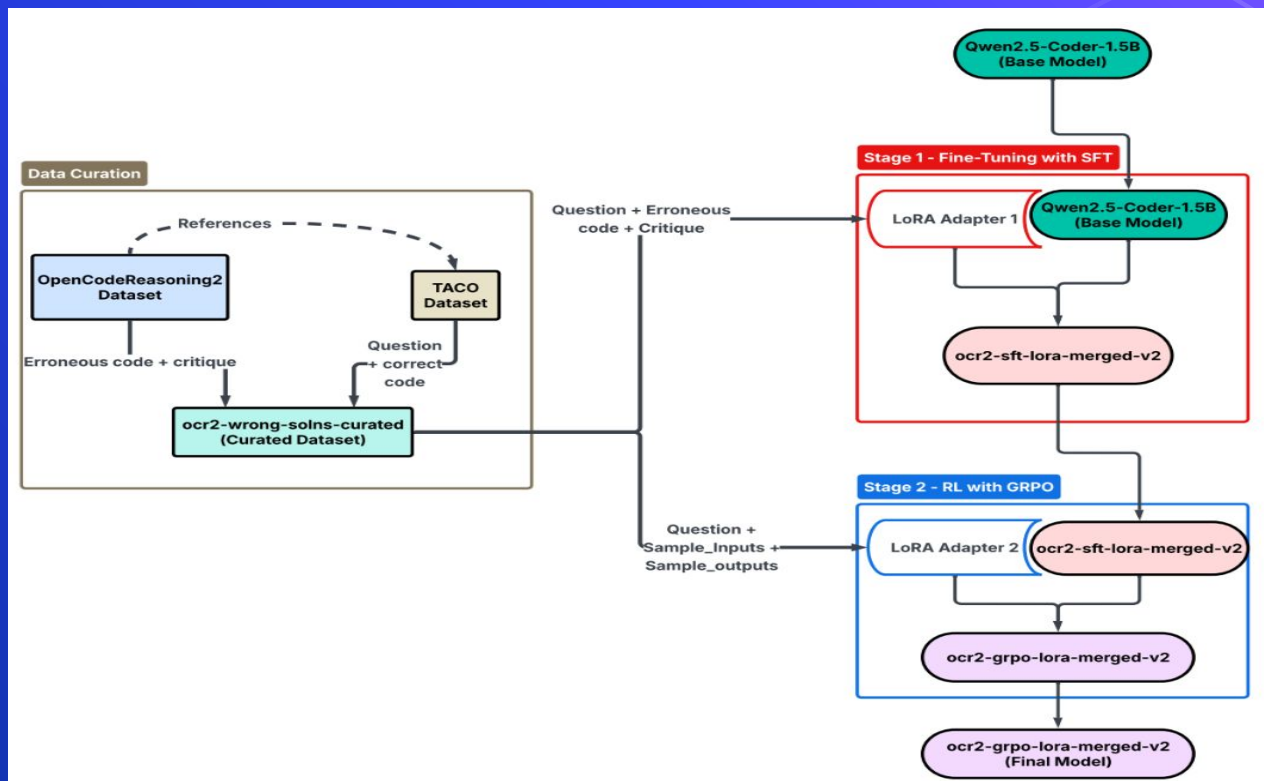
⬡ Training:

- Stage 1 - Supervised Finetuning using QwQ's critique for the **Chain-of-thought generation**
- Stage 2 - GRPO Reinforcement Learning using sample inputs and outputs for **correct code generation**

Execution Pipeline

- Both SFT and RL were performed with QLoRA using Unsloth library.
- At the end of each stage, the LoRA adapters were merged to produce an independent model
- Output models from both stages are available in HuggingFace Hub

Execution Pipeline



Training Setting

Mandatory components are taken from the dataset as follows:

<u>Type</u>	<u>Field</u>	<u>Source</u>
Input	Problem statement	Directly from Taco dataset by merging
Input	Erroneous solution code	Code generated by R1
Output	Chain-of-thought (CoT)	Critique provided by QwQ
Output	Proposed correct code	One randomly selected correct solution in Taco dataset
Output	Validation test cases	No data from the dataset itself. Therefore, it was allowed to emerge as an untrained capability

Training Setting

Intermediary components are taken from the dataset as follows:

<u>Training stage</u>	<u>Component</u>	<u>Source</u>
SFT	User prompt	Standard text: "This code is buggy; fix it."
SFT	System prompt	Standard text: "You are a QA engineer."
GRPO	RL assessment - input	Sample inputs from Taco dataset
GRPO	RL assessment - output	Sample outputs from Taco dataset

Training Cycle

SFT	150 steps Final loss of 0.687	Loss is not given much importance because the response includes code. (SFT loss does not capture code correctness) Allows the model to be exposed to different correct answers.
GRPO	20 steps Final loss of 0.2415	Very small number of steps due to resource constraints. Requires more steps for meaningful training.

Parameters

- ⬡ Training parameters (alpha, rank, etc.) were chosen using **empirical tuning** from standard baseline values.
- ⬡ Important parameters:
 - Lora rank (r) = 16
 - Lora alpha = r
 - Repetition penalty = 1.2

Training stage 1 (SFT)

- Reformatted data into structured chat format with required tags (<CoT>, <proposed_fix>)
- Filtered samples with long CoT sequences (>5000 tokens) for **training stability**
- ~150 training steps performed due to computational constraints
- Intended outcomes:
 - Establish baseline behaviour
 - Expose model to critique-style feedback for buggy code
 - Teach correct code generation for each problem

Training stage 2 (RL)

- ⬡ GRPO was performed with a few reward functions:
 - Presence of required tags (<CoT>, <proposed_code>, etc.)
 - Strictness in tag structure
 - **Execution scores** (number of passed test cases per code generated)
- ⬡ The above rewards were given weightages of **0.15, 0.15, and 0.70** respectively.
- ⬡ 20 training steps performed due to computational constraints

Demo setting

- ⬡ A problem statement involving padding text to form a box was given to the model

“Frame the whole text with characters «» of the minimum size. If a line cannot be aligned perfectly (for example, the line has even length, while the width of the block is uneven), you should place such lines rounding down the distance to the left or to the right edge and bringing them closer left or right alternatively (you should start with bringing left).”*

Result from demo (CoT)

<CoT>

Okay, I need to check if the provided solution is correct for the given problem. Let me start by understanding the problem again.

The task is to center-align each line within a block. The block is framed by two asterisks. If a line's length doesn't match the block's width exactly, we need to adjust the padding so that the line is centered as much as possible. We round down the difference between the line's length and the block's width, then distribute the padding evenly on both sides.

Looking at the solution code:

The code reads all lines from stdin, including handling possible empty lines. Then it calculates the maximum line length. The top_bottom line is built with the appropriate number of '*'.

Then, for each line, it calculates the difference between the line's length and max_length. If

Result from demo (Proposed code)

```
<judgement>wrong</judgement>

<proposed_fix>
import sys

def get_input():
    lines = []
    for line in sys.stdin:
        lines.append(line.strip())
    return lines

def main(lines):
    result = []
    max_length = 0
    for line in lines:
        max_length = max(max_length, len(line))
    result.append('*' * (max_length + 4))
    for line in lines:
        padding = (max_length - len(line)) // 2
        result.append(f'{" " * padding}{line}{" " * padding}*')
    result.append('*' * (max_length + 4))
    return '\n'.join(result)

print(main(get_input()))
```

Result from demo (Test validation)

```
<test_validation>
assert main(['This is', 'Codeforces', 'Beta', 'Round', '5']) == '*****\n* This is *\n
*          *\n*Codeforces*\n*   Beta   *\n* Round   *\n*          5   *\n*****\n'
assert main(['welcome to the', 'Codeforces', 'Beta', 'Round 5', 'and', 'good luck']) == '*****
*****\n*welcome to the*\n* Codeforces *\n*          Beta   *\n*          Round 5   *\n*          and      *\n*          good luck *\n*****'
</test_validation>
```

Observation

- Qualitative improvement observed in response structure and format adherence (<CoT>, <proposed_fix>) following SFT, confirming baseline behaviour was established.
- No significant improvement following GRPO due to very low number of training steps (~20)
- Occasional accurate test case generation was observed despite not being an explicitly trained objective, suggesting **emergent capability**.

Future Improvements

- Utilise a **dataset that tracks changes** made to buggy code
- Extend GRPO training to a meaningful number of steps (~500+) to allow reward signal to take effect
- Experiment with a larger base model (7B+) better suited to complex reasoning tasks
- Investigate whether the emergent test case generation capability scales with further GRPO training

Thank you!

